From General Chatbot to Specialized Assistant: Haive as a Modular, Prompt-Engineered Generative AI Platform

Iris Kate M. Manilag
irismanilag@gmail.com

Jhon Lloyd M. Valencia
jhonlloydval@gmail.com

openIt – Data Science Bootcamp
Manuel S. Enverga University Foundation – Lucena

I. PROJECT OVERVIEW

a. Nature of the Proposed System

ToolHive AI is a modular Generative AI (GenAI) tools library developed as a Streamlit-based web application. Grounded in the growing body of research on modular, task-oriented AI assistant architectures, the platform allows users to discover, launch, and even create their own AI tools through a unified interface, rather than relying on a single general-purpose chatbot (Oelen & Auer, 2025; Armstrong, 2025). Each tool is purpose-built for a specific task or user role, addressing the well-documented limitation of general-purpose chatbots in producing focused, reliable outputs for domain-specific workflows. ~~ Haive is a modular Generative AI (GenAI) tools library developed as a Streamlit-based web application. Rather than offering a single general-purpose chatbot, the platform organizes AI capabilities into focused, task-specific assistants called Hives — each purpose-built for a specific user role or workflow. This approach reflects a growing body of research on modular, task-oriented AI assistant architectures that outperform general-purpose chatbots in domain-specific reliability and output quality (Oelen & Auer, 2025; Armstrong, 2025).

The application is structured around three navigational layers: a branded landing page, a searchable Tools Library dashboard, and individual Hive pages. Users can discover and launch any of the 12 built-in Hives, interact with HAIVE (a general-purpose open chat assistant), or create entirely new tools using the Custom Tool Builder — all without writing a single line of code.

The system is built on Python and Streamlit, powered by locally hosted large language models (LLMs) via Ollama. The default model is phi4-mini, with llama3.2 available as an alternative model through the HAIVE interface. Additional models — including gemma3:4b, qwen3.5, claude-3-5-sonnet, and gemini-2.0-flash — are listed in the model registry as upcoming integrations. All AI inference runs locally, eliminating cloud dependency and protecting user data privacy.

The platform also includes a Retrieval-Augmented Generation (RAG) engine (utils/utils_rag.py) built on ChromaDB and Ollama's nomic-embed-text embedding model. This enables select Hives to ground their AI responses in user-provided reference documents, improving factual accuracy and context-awareness beyond what prompt engineering alone can achieve.

The initial built-in toolkit includes 12 specialized Hives:

HIVE	CATEGORY	TYPE	PURPOSE
HAIVE	General	Multi-turn	Open-ended chat for drafting, studying, planning, and problem solving
INTERVIEW COACH HIVE	Professional	Multi-turn	Mock interview practice with structured feedback
DOC SUMMARIZER HIVE	Academic	Single-turn	Summarizes pasted text into key points and action items
DOC PARAPHRASER HIVE	Academic	Single-turn	Rewrites text in selected tones while preserving meaning
GRADEWISE HIVE	Academic	Multi-turn	Tracks subjects, itemized scores, risk levels, and academic analytics

FORECASTING HIVE	Academic	Multi-turn	Forecasts required scores, targets, risk levels, and study priorities
ROLEPLAY CREATOR HIVE	Education	Multi-turn	Creates a configured AI persona for roleplay conversations
WELLNESS COMPANION HIVE	Wellness	Multi-turn	Provides reflective journaling support for everyday wellbeing
FACT CHECKER HIVE	Media Literacy	Single-turn	Reviews claims, article excerpts, and credibility signals; supports RAG-grounded analysis via user-uploaded reference documents
CAREER ROADMAP HIVE	Professional	Single-turn	Builds skill-gap analyses and 30-60-90 day career plans
GRAMMAR CHECKER HIVE	Academic	Single-turn	Reviews grammar, clarity, punctuation, and writing quality
ESSAY GENERATOR HIVE	Academic	Single-turn	Generates structured essay or article drafts; accepts optional reference notes or pasted context
QUIZ & FLASHCARD GENERATOR HIVE	Education	Single-turn	Creates quizzes and flashcards from notes or source text

Haive is a capstone prototype intended to demonstrate how diverse GenAI workflows can be packaged into reusable, guided tools that serve distinct user needs more effectively than a generic chatbot interface (Zhao et al., 2024; Gao et al., 2024).

b. Problem/Need Addressed

The rapid adoption of generative AI in the Philippines and globally has created both significant opportunity and a critical usability gap. As of 2024, 46% of Filipino workers utilize AI tools at least once a month, surpassing the global average of 39% (JobStreet by SEEK, 2024). Despite this high adoption rate, experts observe that the country has high adoption but low maturity: most users rely on commercial AI services for content-related tasks rather than in-production work that generates real productivity gains (Ligot, as cited in PhilSTAR Life, 2026).

The International Labour Organization (2026) estimates that more than one-quarter of Philippine employment — approximately 12.7 million workers — is exposed to generative AI, representing the highest exposure rate among ASEAN countries with comparable data. Meanwhile, the Department of Trade and Industry launched the National AI Strategy Roadmap (NAISR) 2.0 in July 2024, reflecting generative AI advancements, though the Philippines has yet to enact legally binding AI regulations (UNESCO, 2025).

This gap is compounded by the nature of general-purpose chatbots: they produce unfocused, inconsistent, or poorly structured outputs when approached with specific professional tasks such as interview preparation, grade tracking, document paraphrasing, or fake news detection. Users are expected to craft detailed prompts themselves, which requires AI literacy that most users have not yet developed. The 2024 Microsoft and LinkedIn Work Trend Index found that Filipino employees lead globally in AI adoption for productivity, yet training gaps remain — particularly in using AI effectively for specific roles and functions (Microsoft & LinkedIn, 2024).

Haive addresses this problem by packaging different AI workflows into focused, task-specific Hives. Each Hive has a clear purpose, target user, guided input format, and structured output — reducing the burden on the user to prompt the AI correctly and increasing the reliability and usefulness of responses across professional, academic, and personal use cases.

c. What is a Multi-Tool AI Platform?

Haive is an example of a multi-tool AI platform — a design architecture where multiple specialized AI assistants are unified under a single interface, each scoped to a distinct task or domain. This contrasts with the single-chatbot model, where one assistant handles all requests without task-specific optimization.

This concept is grounded in research on modular AI assistant architectures (Oelen & Auer, 2025), which demonstrate that AI tools designed for specific tasks outperform generic chatbots on three dimensions:

- Output reliability — structured prompts produce more predictable, higher-quality responses.
- User accessibility — guided inputs lower the AI literacy barrier for non-technical users.
- Task completeness — scoped prompts reduce hallucination and off-topic generation.

The Capabara model (Straits Interactive, n.d.) is one practical reference for this pattern: a capability tools library where different AI tools serve different functional roles within one platform. Haive extends this by allowing users to not only use built-in tools but also create and publish their own Hives through the Custom Tool Builder.

In terms of architecture, Haive implements this through:

- A single shared AI client (utils/ollama_client.py) that all Hives call.
- A shared prompt boundary (prompts/tool_scope.txt) that constrains each Hive to its intended scope.
- A tools registry (utils/tools_data.py) that serves as the single source of truth for all built-in Hives.
- A runtime merger that combines built-in Hives and user-created custom tools into one unified library.

The result is a platform where each assistant feels purpose-built, but all tools share the same underlying infrastructure and design system.

d. Prompt Engineering as the Core AI Mechanism

Since Haive does not implement model fine-tuning in its current prototype version, prompt engineering serves as the primary mechanism for AI customization. A Retrieval-Augmented Generation (RAG) engine has also been implemented for select Hives, enabling responses to be grounded in user-provided reference documents (see Section III.G). Each Hive utilizes a carefully designed system prompt that defines the AI's role, behavior, response format, tone, and operational boundaries according to its intended function.

Previous studies have shown that task-specific prompting can significantly improve output reliability, consistency, and usefulness compared to unrestricted prompting approaches (Zhao et al., 2024; Gao et al., 2024). Haive applies this principle through four key prompt engineering strategies.

1. Role Assignment

- Each Hive's system prompt explicitly assigns the AI a professional identity. For example, the Interview Coach Hive is configured to function as a domain-specific interviewer, the Wellness Companion Hive is designed as a non-clinical emotional support companion, and the Fact Checker Hive operates as a credibility analyst. Assigning a role narrows the model's response space and aligns generated outputs with the intended purpose of each Hive.

2. Output Format Specification

- Several Hives require structured outputs to maximize usability. Tools such as the Doc Summarizer Hive, Career Roadmap Hive, and Forecasting Hive include explicit formatting instructions within their system prompts. The AI may be instructed to produce bullet-point summaries, milestone-based career plans, or score forecasts accompanied by explanatory justifications. This approach ensures that generated responses are immediately actionable and require minimal post-processing by users.

3. Shared Prompt Boundary (Scope Enforcement)

- All specialized Hives operate within a shared scope template stored in prompts/tool_scope.txt. This boundary layer prevents a Hive from behaving as a general-purpose chatbot and reinforces task-specific behavior. For example, if a user asks the Interview Coach Hive for cooking advice, the scoped prompt redirects the interaction back to interview-related assistance. The HAIVE assistant is the only component exempt from this restriction because it is intended to support open-ended conversations.

4. Tone and Behavioral Control

- Certain Hives require specific communication styles to meet their objectives. The Wellness Companion Hive is instructed to respond empathetically, avoid overly prescriptive language, and encourage professional support when serious emotional distress is detected, consistent with recommendations from Moylan and Doherty (2025) and Safi et al. (2024). Similarly, the Doc Paraphraser Hive accepts user-selected tones such as formal, casual, academic, or simplified. These preferences are passed as prompt variables, allowing the AI to dynamically adapt its writing style for each request.

Collectively, these strategies allow the complexity of interacting with large language models to be managed internally by the system. Rather than composing detailed prompts themselves, users interact with guided interfaces that encapsulate prompt engineering best practices within each Hive.

e. Key Features and Functionalities

ToolHive AI is organized around three navigational layers: an onboarding page, a Tools Library dashboard, and individual tool pages. The following highlight how the system achieves its objectives:

Landing Page: The Landing Page serves as the branded entry point of the Haive platform. It introduces users to the concept of a modular AI tools library, provides an overview of the platform's capabilities, and highlights the benefits of task-specific AI assistants. The page also showcases selected Hives and guides users toward the Tools Library Dashboard, where all available tools can be explored and launched.

Tools Library Dashboard: The Tools Library Dashboard functions as the primary navigation hub of the system. It displays all available Hives as searchable and filterable cards sourced from both built-in and user-generated tool repositories. Built-in Hives are loaded from

utils/tools_data.py, while custom tools are retrieved from data/custom_tools.json. Each card presents key information, including the Hive name, intended user group, brief description, category label, and a Launch button for direct access. A featured HAIVE card is prominently displayed to provide quick access to the platform's general-purpose assistant. The dashboard also includes an Add New Toolkit button, which opens the Custom Tool Builder interface for creating new Hives.

HAIVE: General Chat Assistant: HAIVE is the platform's open-ended conversational AI assistant. Unlike the specialized Hives, it operates without prompt scope restrictions and is designed to support a broad range of activities, including drafting, studying, brainstorming, planning, and general problem-solving. Users can select between the phi4-mini and llama3.2 language models when interacting with HAIVE. Additional models, including gemma3:4b, qwen3.5, claude-3-5-sonnet, and gemini-2.0-flash, are listed in the model registry as planned future integrations. This flexibility allows HAIVE to function as a general-purpose assistant while remaining integrated within the broader Haive ecosystem.

f. Scope and Boundaries

Haive covers the design, development, and demonstration of a modular AI tools platform using locally hosted large language models. The full user journey, from onboarding to tool discovery, input submission, AI processing, and structured output display, is implemented across all 13 Hives (12 specialized Hives plus HAIVE).

Current boundaries:

- Haive is a capstone prototype and does not include production-grade security, user authentication, or multi-user account management.
- All tools are powered by locally hosted LLMs via Ollama. The platform implements a local RAG engine (ChromaDB + nomic-embed-text) for select Hives. Model fine-tuning and external database querying are not implemented in this version.
- GradeWise computes grade estimates using transparent mathematical formulas rather than a trained machine learning model.
- The Wellness Companion is explicitly not a clinical tool. It does not diagnose, treat, or provide therapeutic interventions. Users in crisis are directed to seek professional support.
- The Fact Checker provides LLM-assisted credibility analysis based on linguistic patterns, logical consistency, and optionally user-provided reference documents through the RAG engine. It is not connected to a live fact-checking database.
- The system requires a local machine with Ollama installed and phi4-mini (or llama3.2) downloaded, together with ChromaDB and nomic-embed-text for RAG features. Internet access is not required during runtime.
- Custom tools are stored in session state or data/custom_tools.json and do not include cloud storage or multi-user synchronization.

g. Intended Users and Stakeholders

Primary users:

- Job applicants and working professionals: Interview Coach Hive and Career Roadmap Hive
- Students and researchers: Doc Summarizer Hive, Doc Paraphraser Hive, GradeWise Hive, Forecasting Hive, Grammar Checker Hive, Essay Generator Hive, and Quiz & Flashcard Generator Hive
- Educators and trainers: Roleplay Creator Hive and Quiz & Flashcard Generator Hive
- General users: Wellness Companion Hive, Fact Checker Hive, and HAIVE

Secondary users:

- Non-technical professionals who want to create their own AI tools through the Custom Tool Builder without writing code.

Key stakeholders:

- Bootcamp instructors and evaluators assessing the project as a capstone deliverable
- Prospective employers or development teams that may adopt or extend the platform
- The broader community of Filipino AI practitioners aligned with the national AI upskilling agenda (Lumify Group, 2025; DTI, 2024)

h. Expected Benefits

- **For Job Applicants and Professionals:** Structured, role-specific interview practice and career planning with actionable AI-generated feedback, reducing reliance on expensive coaching services.

- **For Students and Researchers:** Fast structured summarization, tone-aware paraphrasing, real-time grade tracking, required score forecasting, and writing support tools that improve academic productivity.

- **For Educators and Trainers:** Configurable AI roleplay personas and quiz/flashcard generation that support scenario-based learning and rapid content creation.

- **For General Users:** A judgment-free space for emotional reflection through the Wellness Companion, and a practical tool for evaluating news credibility through the Fact Checker. Both address growing concerns related to mental wellbeing and misinformation (Safi et al., 2024; Polu, 2024).

- **For Non-Technical Users:** The Custom Tool Builder enables anyone to create and launch a specialized AI assistant without coding knowledge.

- **For the Philippine AI Ecosystem:** Haive demonstrates a practical, low-cost, privacy-preserving GenAI application that runs entirely on local hardware using open-source models. This approach remains accessible even in resource-constrained settings and aligns with the goals of NAISR 2.0 (DTI, 2024).

II. CORE DATA REQUIREMENTS

a. Major System Modules and Workflows

1. Landing Page Module

- Entry point rendered when the application loads. Introduces Haive, displays brand visuals, previews available Hives, and routes users to the Tools Library.

2. Tools Library Dashboard Module

- Main navigation hub. Loads built-in tool definitions from utils/tools_data.py and custom tools from data/custom_tools.json. Renders filterable and searchable Hive cards. Routes users to the selected Hive when a card is launched.

3. HAIVE General Chat Module

- Open-ended multi-turn AI chat. Supports model selection and maintains conversation history in Streamlit session state. No scope constraints are applied.

4. Interview Coach Module

- Multi-turn conversational interview practice tool. Accepts role or domain selection, generates role-specific interview questions, evaluates responses, and returns structured feedback with improvement suggestions.

5. Doc Summarizer Module

- Single-turn text processing tool. Accepts long-form text and returns a structured summary, key points in bullet format, and action items.

6. Doc Paraphraser Module

- Single-turn tone-aware rewriting tool. Accepts text input and tone selection (formal, casual, academic, or simplified). Returns a rewritten version that preserves the original meaning. Displays the original and paraphrased text side by side.

7. GradeWise Module (Grade Predictor Page)

- Multi-turn academic tracking tool. Accepts subject management data, term weights, assessment categories, and itemized scores. Computes current weighted grade, required score, passing probability, risk level, and GWA equivalent. Includes analytics charts, a radar chart, and a risk map. Supports forecasting chat when Forecasting session data is available.

8. Forecasting Hive Module

Multi-turn academic forecasting tool. Accepts current grade, completed weight, passing grade target, and recent performance trend. Returns the required remaining average, passing probability, and an AI-generated action plan.

9. Roleplay Creator Module

- Multi-turn persona-based conversation tool. Accepts persona configuration, including name, role, background, and behavioral tone. Builds a system prompt from the provided configuration, maintains conversation memory in session state, and generates in-character responses.

10. Wellness Companion Module

- Multi-turn emotional support tool. Displays a non-clinical disclaimer upon entry. Accepts open-ended emotional or reflective input and returns empathetic, supportive, and non-prescriptive responses. May optionally provide journaling prompts.

11. Fact Checker Module

- Single-turn credibility analysis tool with optional RAG grounding. Users may load reference documents into a local ChromaDB knowledge base through a collapsible panel before submitting a claim. Accepts a news headline, claim, or article excerpt. Retrieves relevant knowledge base content when available and injects it into the system prompt. Returns a structured credibility assessment (Likely Reliable, Unclear, or Likely Misleading), reasoning analysis, red flags, and a verification checklist.

12. Career Roadmap Module

- Single-turn career planning tool. Accepts the user's current role, existing skills, and target career goal. Returns a structured roadmap containing skill gaps, 30-60-90 day milestones, and prioritized learning resources.

13. Grammar Checker Module

- Single-turn writing quality review tool. Accepts pasted text and returns corrections and explanations covering grammar, clarity, punctuation, and overall writing quality.

14. Essay Generator Module

- Single-turn essay drafting tool. Accepts a topic, essay type (Argumentative, Expository, Analytical, Reflective, Compare & Contrast, or Blog Post/Article), tone, and approximate target length. Users may optionally provide reference notes or source text, which are included as contextual information in the prompt. Returns a structured essay draft consisting of a title, introduction, body sections, and conclusion.

15. Quiz & Flashcard Generator Module

- Single-turn study aid generation tool. Accepts notes or source text and returns quiz questions with answer keys or flashcard sets.

16. Custom Tool Builder Module

- Form-based tool creation module. Accepts a tool name, description, target user, category, system prompt, input placeholder, and output format instruction. Saves tool definitions to data/custom_tools.json. Newly created tools appear immediately in the Tools Library.

b. Key Data Requirements

Component	Purpose	Implementation
Python 3.10+	Core programming language	Required runtime (developed on 3.12)
Streamlit	Multi-page web application framework	Pages folder + session state
Ollama	Local LLM inference engine	REST API at localhost:11434
phi4-mini	Default primary language model	Pulled via Ollama CLI
llama3.2	Secondary available model (HAIVE)	Pulled via Ollama CLI
nomic-embed-text	Embedding model for RAG	Pulled via Ollama CLI
ChromaDB	Local vector store for RAG	pip install chromadb
Requests	HTTP client for Ollama API calls	Used in ollama_client.py and utils_rag.py
Plotly	Analytics charts and visualizations	Used in GradeWise dashboard
Custom CSS	UI design system	Injected through Streamlit via ui.py
JSON	Custom tool and data storage	data/custom_tools.json
Streamlit Session State	Conversation memory across pages	st.session_state

III. SYSTEM ARCHITECTURE AND AI WORKFLOW

a. Three-Layer Architecture

Haive uses a modular multi-page Streamlit architecture organized into three layers.

Layer 1: Landing Page
Haive/app.py

- The branded entry point. Introduces the platform, displays visual previews of available Hives, and directs users to the Tools Library. It establishes user expectations before any AI interaction begins.

Layer 2: Tools Library Dashboard
Haive/pages/0_Tools_Library.py

- The central navigation and discovery interface. Loads tool definitions from the built-in registry and custom tools file, renders them as filterable Hive cards, displays the featured HAIVE card, and routes users to individual Hive pages upon launch.

Layer 3: Individual Hive Pages
Haive/pages/[1–13]_*.py

- Each Hive is implemented as a dedicated Streamlit module with its own logic, interface, and output layout. Prompt engineering is handled internally, allowing users to interact through guided inputs rather than raw prompt boxes. Shared services, including the Ollama client and UI helper functions, are centralized within the utils/ directory.

b. Project Structure

```
openiT-Data-Science-Bootcamp/
├── README.md
└── Haive/
    ├── app.py
    ├── data/
    │   ├── custom_tools.json
    │   └── chroma_db/
    ├── files/
    │   ├── Haive_Paper.pdf
    │   ├── Haive_Paper.docx
    │   └── haive_brand_guide.html
    ├── pages/
    │   ├── 0_Tools_Library.py
    │   ├── 1_Interview_Coach.py
    │   ├── 2_Document_Summarizer.py
    │   ├── 3_Document_Paraphraser.py
    │   ├── 4_GradeWise.py
    │   ├── 5_Roleplay_Creator.py
    │   ├── 6_Wellness_Companion.py
    │   ├── 7_Fact_Checker.py          ← RAG-enabled
    │   ├── 8_Career_Roadmap.py
    │   ├── 9_Custom_Tool_Runner.py
    │   ├── 10_Grammar_Checker.py
    │   ├── 11_Essay_Generator.py
    │   ├── 12_Quiz_Flashcard_Generator.py
    │   ├── 13_Forecasting.py
    │   ├── about.py
    │   ├── haive.py
    │   └── sources.py
    ├── prompts/
    │   └── tool_scope.txt
    └── utils/
        ├── __init__.py
        ├── ollama_client.py
        ├── tools_data.py
        ├── utils_rag.py               ← RAG engine
        └── ui.py
```

c. AI Inference Architecture

All Hives share a unified AI inference architecture powered by locally hosted large language models served through

Ollama. Communication occurs through Ollama's REST API at localhost:11434/api/chat. The default model is phi4-mini, while llama3.2 is available as an alternative model through HAIVE.

Using a locally deployed model provides three key advantages for this prototype:

1. **No cloud API costs** — eliminates operational expenses during development and use
2. **Data privacy** — all processing occurs on the user's machine; no user data is sent to external servers
3. **Offline runtime** — the platform remains fully functional without internet connectivity once set up (Jabarian, 2024; Belsare, 2024)

Each Hive interacts with a shared utils/ollama_client.py module, which handles Ollama chat requests, default model configuration, available model metadata, prompt template loading, scoped system prompt construction, and Ollama health checks. The tool module constructs a prompt using user inputs and predefined system instructions, sends the request to the Ollama API, and displays the generated response using an output format tailored to the Hive's purpose.

d. Prompt Engineering Strategy

Prompt engineering is the primary AI customization mechanism in Haive. Each Hive uses a structured system prompt combining role assignment, output format specification, shared scope enforcement, and tone or behavioral control. The four strategies are explained in detail in Section I.D above.

The specific prompt design for each Hive is summarized below:

HAIVE — No scope constraint. Open-ended system prompt defines a helpful general assistant identity. Supports model switching.

Interview Coach — Configured as a professional interviewer for a user-selected role or industry. Asks one question at a time, evaluates responses based on clarity, relevance, and depth, and provides structured improvement suggestions before proceeding.

Doc Summarizer — Instructs the model to return a concise summary, key points in bullet format, and action items where applicable. Emphasizes brevity and discourages unnecessary commentary.

Doc Paraphraser — Rewrites user-provided text according to a selected tone (formal, casual, academic, or simplified). Preserves original meaning while modifying vocabulary, sentence structure, and style.

GradeWise — Provides AI-powered insights as a companion to the mathematical grade computation system. Interprets grade data and generates personalized study recommendations.

Forecasting Hive — Analyzes academic scenario inputs and generates a required score, probability estimate, and structured action plan. Can integrate GradeWise session context when available.

Roleplay Creator — Dynamically constructs an AI persona from user-defined settings such as name, role, background, and behavioral tone. Instructs the model to remain fully in character throughout the interaction.

Wellness Companion — Defined as a non-clinical emotional support companion. It is explicitly restricted from diagnosing conditions or simulating therapy. It responds empathetically, asks reflective questions, and encourages professional support when serious distress is detected (Moylan & Doherty, 2025; Safi et al., 2024).

Fact Checker — Evaluates submitted content using linguistic credibility indicators, logical consistency, and general knowledge alignment. When reference documents are ingested via the RAG panel, retrieved chunks are appended to the system prompt and the model is instructed to explicitly evaluate whether the retrieved content supports, contradicts, or is irrelevant to the claim. It produces a structured credibility assessment (Likely Reliable, Unclear, or Likely Misleading) with reasoning, red flags, and verification recommendations (Raychaudhuri et al., 2024; Polu, 2024).

Career Roadmap — Analyzes the user's current role, skills, and career objective. Generates a structured plan including skill gaps, 30-60-90 day milestones, and prioritized learning resources.

Grammar Checker — Reviews pasted text for grammar, clarity, punctuation, and writing quality. Returns corrections with brief explanations.

Essay Generator — Generates a structured draft based on topic, essay type, tone, and target word count. When reference notes or source text are provided, they are appended as contextual input for the model; otherwise, the essay is generated from general knowledge.

Quiz & Flashcard Generator — Creates quizzes with answer keys or flashcard sets from user-submitted notes or text.

e. Conversation Memory Architecture

Three Hives support multi-turn conversations: Interview Coach, Roleplay Creator, and Wellness Companion. HAIVE also maintains multi-turn memory. These Hives use Streamlit session state (st.session_state) to preserve conversation history during runtime.

When the user submits a new message, the complete message history is passed to the Ollama API as a structured messages array, enabling the model to maintain contextual continuity and reference earlier parts of the interaction.

The remaining Hives operate as single-turn systems. Each request is processed independently without retaining prior interactions, which is appropriate for their task-oriented input-output workflows.

f. RAG Architecture

Haive implements a local Retrieval-Augmented Generation (RAG) engine in utils/utils_rag.py. This engine allows select Hives to ground AI responses in user-provided reference documents rather than relying solely on the LLM's parametric knowledge.

Components:

- Vector store: ChromaDB (PersistentClient) stored at data/chroma_db/. Each Hive that uses RAG maintains its own named collection (e.g., "fact_checker").
- Embedding model: Ollama's nomic-embed-text, accessed via the /api/embeddings endpoint at localhost:11434.
- Chunking: Documents are split into 500-character chunks with 50-character overlap before embedding, improving retrieval coverage for longer texts.

Public API exposed by utils_rag.py:

FUNCTION	DESCRIPTION
INGEST_TEXT(COLLECTION, TEXT, DOC_ID)	Chunk, embed, and store a document string
INGEST_FILE(COLLECTION, FILE_PATH, DOC_ID)	Read a text file and ingest it
RETRIEVE(COLLECTION, QUERY, TOP_K=3)	Return the top-K most relevant chunks as a string
COLLECTION_EXISTS(COLLECTION)	Check whether a collection has ingested documents
DELETE_COLLECTION(COLLECTION)	Remove a collection for re-ingestion

RAG workflow in the Fact Checker Hive:

1. The user optionally pastes reference material and clicks "Add to knowledge base." The text is chunked, embedded, and stored in ChromaDB under the "fact_checker" collection.
2. When a claim is submitted, retrieve() embeds the query and returns the top-3 most relevant chunks.
3. If chunks are found, they are appended to the system prompt under a "Retrieved reference context" block. The AI is instructed to evaluate whether the retrieved content supports, contradicts, or is irrelevant to the claim.
4. If no documents have been ingested, the Hive falls back to standard prompt-only analysis.

The RAG engine is designed to be tool-agnostic. Any Hive can call ingest_text() and retrieve() with its own collection name. The Fact Checker Hive is the first integration, but the architecture supports extension to other Hives such as a document-grounded Career Roadmap or a citation-aware Essay Generator in future iterations.

g. Custom Tool Execution Architecture

User-generated Hives are stored as structured JSON objects in data/custom_tools.json. Each entry contains the tool name, description, target user, category tag, system prompt, input placeholder text, and output format instruction.

When a user launches a custom Hive from the dashboard, pages/9_Custom_Tool_Runner.py loads the saved configuration, initializes the session using the stored system prompt, and processes all interactions through the same Ollama client used by built-in Hives. This enables an unlimited number of user-created AI tools to be built and executed without modifying the application code.

IV. REFERENCES

[1] Ahmed, S., et al. (2024). Student performance prediction using machine learning algorithms. Applied Computational Intelligence and Soft Computing. https://onlinelibrary.wiley.com/doi/10.1155/2024/4067721

[2] Al-alshaqi, O., et al. (2024). A comprehensive framework for fake news detection combining analysis of text, images, and videos through machine learning and deep learning methods. Repository of RIT Theses. https://repository.rit.edu/cgi/viewcontent.cgi?article=13302&context=theses

[3] Department of Trade and Industry (DTI). (2024). National AI Strategy Roadmap (NAISR) 2.0. https://dict.gov.ph/

[4] Gao, Y., et al. (2024). Retrieval-augmented generation for large language models: A survey. arXiv. https://arxiv.org/abs/2312.10997

[5] International Labour Organization (ILO). (2026). Generative AI and jobs in the Philippines: Labour market exposure and policy implications. https://www.ilo.org/publications/generative-ai-and-jobs-philippines-labour-market-exposure-and-policy

[6] Jabarian, M. (2024, December 10). Building local LLMs app with Streamlit and Ollama. Medium. https://medium.com/@maximejabarian/building-a-local-llms-app-with-streamlit-and-ollama-llama3-phi3-511d519c95fe

[7] JobStreet by SEEK. (2024). Decoding global talent report 2024. https://www.jobstreet.com.ph/

[8] Lumify Group. (2025). Transforming the Philippine workforce: The national AI strategy and AI skills development. Learning News. https://learningnews.com/news/lumifygroup/2025/transforming-the-philippine-workforce-the-national-ai-strategy-and-ai-skills-development

[9] Microsoft & LinkedIn. (2024). 2024 Work Trend Index: AI use at work in the Philippines. https://news.microsoft.com/en-ph/2024/05/23/microsoft-and-linkedin-release-2024-work-trend-index-on-ai-use-at-work-in-the-philippines/

[10] Moylan, K., & Doherty, K. (2025). Expert and interdisciplinary analysis of AI-driven chatbots for mental health support: Mixed methods study. Journal of Medical Internet Research. https://www.jmir.org/2025/1/e67114

[11] Oelen, A., & Auer, S. (2025). TIB AIssistant: A platform for AI-supported research across research life cycles. ISWC 2025 Companion Volume. https://arxiv.org/pdf/2512.16442

[12] PhilSTAR Life. (2026, February 5). How will Filipinos use AI in 2026? Tech experts weigh in. https://philstarlife.com/geeky/491645-ai-technology-2026-predictions

[13] Polu, O. R. (2024). AI-based fake news detection using NLP. International Journal of Artificial Intelligence & Machine Learning, 3(2), 231. https://iaeme.com/MasterAdmin/Journal_uploads/IJAIML/VOLUME_3_ISSUE_2/IJAIML_03_02_019.pdf

[14] Raychaudhuri, S., et al. (2024). Machine learning-based false information analysis tool employing NLP and deep learning. Repository of RIT Theses. https://repository.rit.edu/cgi/viewcontent.cgi?article=13302&context=theses

[15] Rico-Juan, J. R., et al. (2024). Predicting end-of-course marks using LMS usage and personality traits in computer engineering students. Frontiers in Education. https://www.frontiersin.org/journals/education/articles/10.3389/feduc.2025.1562586/full

[16] Safi, A., et al. (2024). AI chatbots for mental health: A scoping review of effectiveness, feasibility, and applications. Applied Sciences, 14(13), 5889. https://www.mdpi.com/2076-3417/14/13/5889

[17] Straits Interactive Pte Ltd. (n.d.). Capabara: Capability tools library. https://www.straitsinteractive.com/

[18] UNESCO Global AI Ethics and Governance Observatory. (2025). Philippines country profile. https://www.unesco.org/ethics-ai/en/philippines

[19] Zhao, S., et al. (2024). Retrieval augmented generation (RAG) and beyond: A comprehensive survey on how to make your LLMs use external data more wisely. arXiv. https://arxiv.org/abs/2409.149